Gradient Boosting Regression of Cortical Bone Modulus — Methods in Detail

BME 5650: Biosolid Mechanics, HW 7 | Companion notebook: `gbr_split.ipynb` (same layout and outputs as the class `currey.ipynb` example)

1. What the notebook does

The notebook `gbr_split.ipynb` is a drop-in replacement for the Currey nonlinear-regression example. It keeps the same structure (one modelling cell, one plotting block), the same data handling and the same outputs, but swaps the physical model $E = k \cdot Ca^{x} \cdot V_f^{y}$ for scikit-learn's `GradientBoostingRegressor`. Because the shuffle and split code are identical (NumPy generator, seed 0, 25% test), both models are trained on exactly the same 277 samples and tested on exactly the same 92, so their numbers can be compared directly.

Step	Currey example (currey.ipynb)	Gradient boosting (gbr_split.ipynb)
Load / clean	read_csv, drop rows missing Ca, Vf or E → n = 369	identical
Inputs	Ca converted to g/g (÷1000), Vf	identical (stacked into a 277×2 matrix)
Split	np.random.default_rng(0).permutation; first 92 = test, remaining 277 = train	identical
Fit	scipy curve_fit, p0 = [500, 2, 3]	GridSearchCV over 384 GBR settings, 5-fold CV on training data, then refit
Printed parameters	k, x, y ± standard error	chosen hyperparameters, CV RMSE ± SD, feature importances
Metrics table	R^2, RMSE, MAE, bias for train and test	identical function and format
Files written	nls_split_params.csv, nls_split_predictions.csv, nls_split.png	gbr_split_params.csv, gbr_split_predictions.csv, gbr_split.png
Figure	parity plot (train grey / test orange) + test residuals	identical layout, colours and labels

Table 1. Side-by-side correspondence between the example notebook and the gradient boosting notebook.

2. Results on the shared 75:25 split

Model	Set	R^2	RMSE (GPa)	MAE (GPa)	bias (GPa)
Currey power law (curve_fit)	train	0.686	4.371	3.313	-0.019
	test	0.717	4.270	3.213	0.035
GBR, sklearn defaults	train	0.924	2.147	1.677	0.000
	test	0.667	4.633	3.222	0.401
GBR, tuned (notebook)	train	0.835	3.168	2.406	0.078
	test	0.727	4.197	3.027	0.383

Table 2. Same 277 / 92 split for all models. Currey parameters: k = 572.7 GPa, x = 2.505, y = 2.300 (identical to currey.ipynb). Tuned GBR: n_estimators = 50, learning_rate = 0.2, max_depth = 2, min_samples_leaf = 10, subsample = 0.7; 5-fold CV RMSE = 4.00 ± 0.4 GPa.

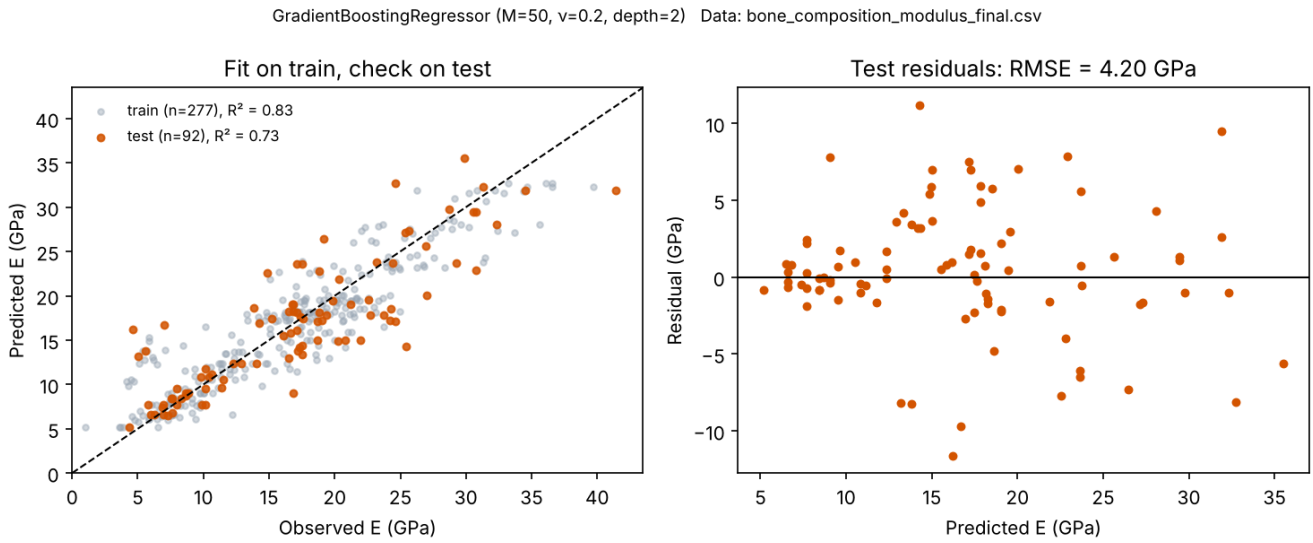


Figure 1. Notebook output (gbr_split.png). Left: observed vs. predicted E; train in grey, test in orange. Right: test residuals.

Reading the results. The tuned gradient boosting model reaches test $R^2 = 0.727$ and RMSE = 4.20 GPa, almost the same as the 3-parameter Currey model ($R^2 = 0.717$, RMSE = 4.27 GPa). The gap between its train and test R^2 (0.83 vs 0.73) shows some over-fitting, which is much worse with the default settings (train 0.92, test 0.67: the defaults do *worse* than Currey on test data). GBR has a small positive test bias (+0.38 GPa: on average it slightly under-predicts) while Currey's bias is about zero. Conclusion: with only Ca and Vf as inputs, a flexible model adds very little over a good physical model. Most of the remaining scatter comes from specimen groups that have different E at the same composition.

3. Gradient boosting, step by step

3.1 The idea

Boosting combines many weak models into one strong one by fitting them *one after another*, with each new model trained to fix the mistakes of all the models before it. In gradient boosting the weak models are small regression trees, and "fixing the mistakes" means fitting the *negative gradient of the loss*, which turns the algorithm into gradient descent in the space of functions (Friedman, 2001). The final model is a weighted sum:

$$F_M(x) = F_0 + \nu \cdot \Sigma_{m=1\ldots M}\, h_m(x), \quad \text{with } x = (Ca, V_f)$$

where F_0 is a constant starting guess, h_m is the m-th tree, M is `n_estimators` and ν is `learning_rate`. Compare this with Currey's model: there the shape is fixed and three numbers are fitted; here the shape itself is built up piece by piece from the data.

3.2 The algorithm (squared-error loss, as used here)

scikit-learn's default loss is `'squared_error'`, implemented as $L(y, F) = \tfrac{1}{2}(y - F)^2$. With $n = 277$ training samples (x_i, y_i):

1. **Initialise.** $F_0 = \text{argmin}_c \Sigma\, L(y_i, c) = \text{mean}(y)$. sklearn does this with an internal `DummyRegressor(strategy="mean")` (stored as `init_`). Here F_0 = 16.861 GPa, the mean training modulus.

2. **For each stage m = 1, 2, …, M:**

   (a) Pseudo-residuals. $r_{im} = -\partial L/\partial F$ *evaluated at* $F_{m-1}(x_i) = y_i - F_{m-1}(x_i)$. For squared error the negative gradient is simply the ordinary residual: how far each specimen's measured E is from the current prediction.

(b) Subsample. Because `subsample` = 0.7 < 1, a random 70% of training rows (193 of 277), drawn without replacement, is used for this stage only (stochastic gradient boosting, Friedman 2002).

(c) Fit a tree. A `DecisionTreeRegressor` with depth ≤ `max_depth` = 2 and at least `min_samples_leaf` = 10 samples per leaf is fit to the pseudo-residuals of those rows (Section 3.3). This splits the (Ca, Vf) plane into rectangles R_{jm} (at most $2^{\text{depth}} = 4$ leaves).

(d) Leaf values. Each leaf gets the value γ_{jm} that minimises the loss in that rectangle. For squared error this is just the *mean residual* of the samples in the leaf, which is exactly what the tree already stores, so no extra line search is needed. (For other losses such as Huber or absolute error, sklearn recomputes leaf values in this step.)

(e) Update with shrinkage. $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$. Only a fraction ν of each correction is applied, so later trees still have work to do. This makes the model generalise better.

3. **Stop** after M = 50 trees. Prediction for a new specimen = F_0 + ν × (sum of the leaf values it lands in).

3.3 How each tree chooses its splits

A regression tree is grown greedily from the top down. At each node, for each input (Ca and Vf), every threshold halfway between neighbouring sorted values is tried, and the split with the largest impurity decrease is kept. sklearn's default `criterion='friedman_mse'` scores a split of a node into left/right children by Friedman's improvement:

$$\text{improvement} = \left(n_L \cdot n_R / (n_L + n_R)\right) \cdot (\bar{r}_L - \bar{r}_R)^2$$

This favours splits that separate the residuals into two groups with very different means and similar sizes. Splitting stops when the depth limit is reached, or when a split would leave fewer than `min_samples_leaf` samples in a child. Figure 2 shows the first tree of the fitted model.

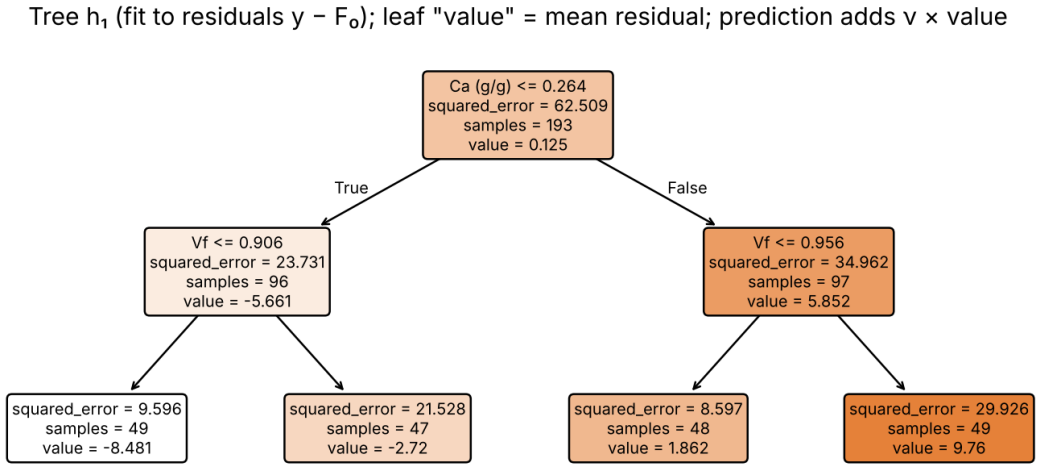


Figure 2. Tree h_1 of the fitted model (via sklearn.tree.plot_tree). It was fit to residuals y − F_0 of a 193-sample subsample. The first split is on Ca, then Vf on each side. 'value' is the mean residual in the node; the root value (0.125) is not exactly zero because the subsample mean differs slightly from the full training mean. Leaf values range from −8.48 GPa (low Ca, low Vf) to +9.76 GPa (high Ca, high Vf): the first tree has already learned that E rises with both inputs.

In text form (`sklearn.tree.export_text`), with the thresholds as fitted:

```
|--- Ca (g/g) <= 0.2640
|   |--- Vf <= 0.9060
|   |   |--- value: [-8.4810]
|   |--- Vf >  0.9060
|   |   |--- value: [-2.7204]
|--- Ca (g/g) >  0.2640
|   |--- Vf <= 0.9561
|   |   |--- value: [1.8624]
|   |--- Vf >  0.9561
|   |   |--- value: [9.7599]
```

Because each tree has depth 2, it can represent an interaction between Ca and Vf (a Vf split inside a Ca split). Depth 1 ("stumps") would give a purely additive model $f(Ca) + g(Vf)$. That is the tree equivalent of Currey's model on a log scale, since log E = log k + x·log Ca + y·log Vf is additive. Across the 50 trees there are only 3–4 leaves each.

3.4 Worked example: how one prediction is assembled

The first test specimen has Ca = 0.2559 g/g, Vf = 0.9654 and measured E = 20.83 GPa. The model's prediction is built up as:

Term	Value (GPa)	Running total (GPa)
F_0 (training mean)	16.861	16.861
$\nu \cdot h_1(x)$ (tree 1, leaf: Ca ≤ 0.264, Vf > 0.906 → −2.720 × 0.2)	-0.544	16.317
$\nu \cdot h_2(x)$	-0.386	15.930
$\nu \cdot h_3(x)$	-0.360	15.570
$\nu \cdot \Sigma\, h_4 \ldots h_{50}(x)$ (47 more trees)	-0.615	14.956
Final prediction $F_{50}(x)$ (= gbr.predict)		14.956

Table 3. Decomposition of one prediction. This specimen is near the Ca split threshold with high Vf, so the trees pull its prediction below the mean. Its measured E (20.83 GPa) is higher than the prediction: it is one of the larger test residuals, a specimen that is stiffer than its composition suggests.

3.5 Watching the model build up

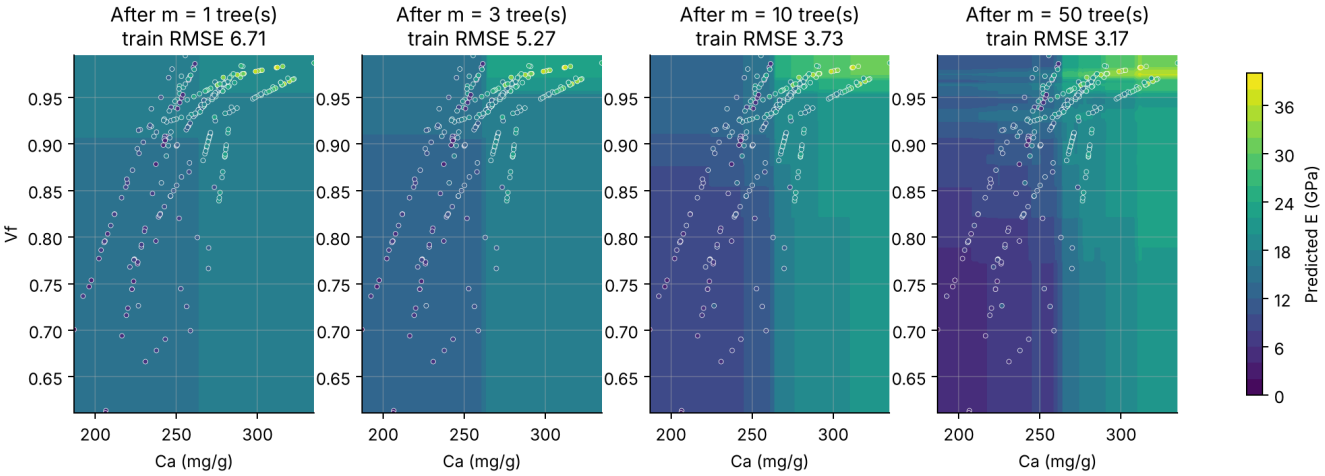


Figure 3. Prediction surface over the (Ca, Vf) training range after 1, 3, 10 and 50 trees (staged_predict), with the training data overlaid in the same colour scale. One tree gives four flat rectangles; each extra tree adds more axis-aligned steps. The final model is a fine 'staircase' that rises toward high Ca and high Vf.

Trees m	1	2	3	5	10	25	50
Train RMSE	6.71	5.91	5.27	4.44	3.73	3.37	3.17
Test RMSE	6.98	6.15	5.56	4.71	4.15	4.11	4.20

Table 4. RMSE (GPa) after m trees. (The training-set SD of E, i.e. the RMSE of F_0 alone, is ≈ 7.8 GPa.)

3.6 Shrinkage, subsampling and over-fitting

Training error can always be pushed down by adding trees, but test error cannot. Figure 4 continues the tuned model to 1000 trees: training RMSE falls to 1.14 GPa while test RMSE rises to 4.59 GPa (its minimum is at about 21 trees). The settings that control over-fitting are:

- **learning_rate ν** — smaller steps; needs proportionally more trees, but each tree's noise matters less.
- **n_estimators M** — the number of steps; too many memorises noise.
- **max_depth, min_samples_leaf** — limit how detailed each tree can be.
- **subsample** — each tree sees a random subset of rows, adding randomness that reduces variance (like bagging). It also makes the *out-of-bag* (OOB) samples available: sklearn stores the loss improvement on each stage's unused rows in `oob_improvement_`. This is a built-in estimate of when extra trees stop helping (Figure 4, right).

Other built-in options not used here: `n_iter_no_change` / `validation_fraction` (early stopping on an internal validation set), `max_features` (random feature subsets per split) and `ccp_alpha` (cost-complexity pruning).

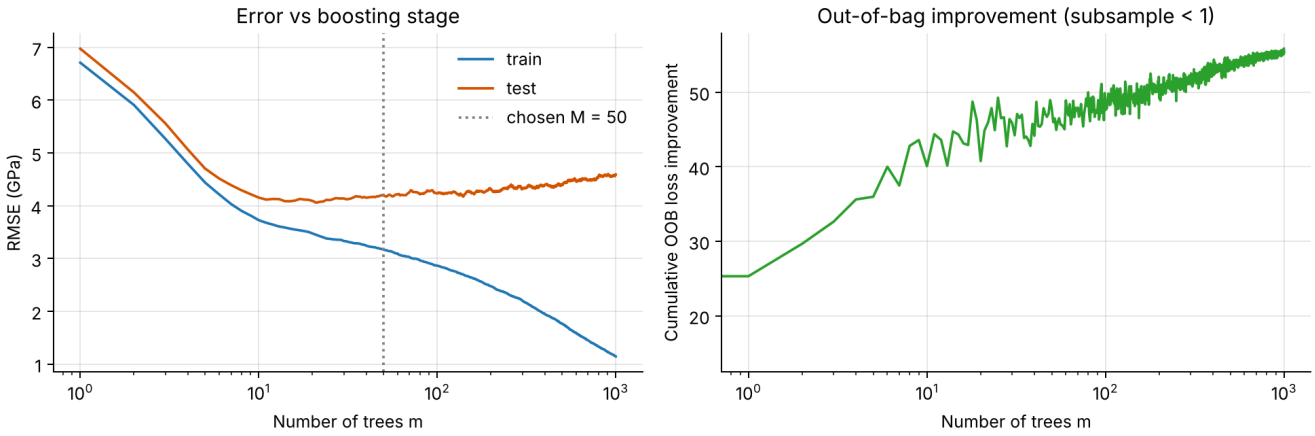


Figure 4. Left: train/test RMSE vs. number of trees (staged_predict) for the tuned settings extended to 1000 trees; dotted line = chosen M. Right: cumulative OOB loss improvement, which flattens once extra trees stop helping.

3.7 Feature importance

The `feature_importances_` attribute is the *mean decrease in impurity*: for each input, add up the squared-error reduction (weighted by the number of samples) from every split on that input, across all trees, then normalise so the importances sum to 1. The fitted model gives **Ca = 0.663, Vf = 0.337**: calcium content explains about twice as much of the variation as volume fraction. This matches the stronger raw correlation of E with Ca (r ≈ 0.81 vs 0.68).

3.8 Properties that follow from the algorithm

- **No normalisation needed.** Splits depend only on the order of the values, so rescaling Ca (mg/g vs. g/g) or Vf changes nothing. The ÷1000 conversion is kept only to match the Currey notebook.
- **Piecewise-constant output.** The prediction is a sum of step functions: no smooth curve, and no meaningful derivative ∂E/∂Ca.
- **No extrapolation.** Beyond the largest (or smallest) split threshold every tree returns a constant, so predictions are flat outside the training range. Currey's power law, by contrast, keeps following its exponents.
- **Interactions are learned automatically**, up to order max_depth.

- **Deterministic given random_state.** `random_state=SEED` fixes the subsampling and the random order in which features are considered, so the notebook reproduces exactly.

4. Hyperparameter selection

Unlike curve_fit, which solves for k, x, y directly, the GBR settings are not learned from the loss. They are chosen by **cross-validation** using `GridSearchCV`:

- The grid has 4 (n_estimators) × 4 (learning_rate) × 4 (max_depth) × 3 (min_samples_leaf) × 2 (subsample) = **384 combinations**.
- `KFold(5, shuffle=True, random_state=0)` divides the 277 training rows into 5 folds of 55–56. Each combination is trained on 4 folds and scored on the 5th, rotating through all 5: **1920 fits** in total, run in parallel with `n_jobs=-1`.
- `scoring='neg_root_mean_squared_error'` — sklearn always maximises scores, so RMSE is negated. The notebook flips the sign back when printing `best_score_` (4.00 GPa).
- The best combination is then **refit on all 277 training rows** (`refit=True`, the default), available as `best_estimator_`.
- **The 92 test rows are never used in tuning**, so the test metrics are an honest estimate of performance on new specimens. The CV RMSE (4.00) and test RMSE (4.20) agree well.

Parameter	sklearn default	Grid searched	Chosen	Meaning
loss	'squared_error'	—	'squared_error'	$L = ½(y−F)^2$; residuals are the negative gradient
n_estimators	100	50, 100, 200, 400	50	number of boosting stages / trees M
learning_rate	0.1	0.01, 0.05, 0.1, 0.2	0.2	shrinkage ν applied to each tree
max_depth	3	1, 2, 3, 4	2	max tree depth; sets max interaction order
min_samples_leaf	1	1, 5, 10	10	minimum samples in any leaf
subsample	1.0	0.7, 1.0	0.7	fraction of rows per tree (<1 = stochastic GB)
criterion	'friedman_mse'	—	'friedman_mse'	split quality score (Section 3.3)
init	None → mean	—	mean of y	starting model F_0
min_samples_split	2	—	2	minimum samples needed to split a node
max_features	None (all)	—	None	features considered per split
max_leaf_nodes	None	—	None	alternative to max_depth
n_iter_no_change	None	—	None	early stopping off
ccp_alpha	0.0	—	0.0	no pruning
random_state	None	—	0	reproducible subsampling

Table 5. GradientBoostingRegressor parameters: defaults, values searched, and values used.

5. Every function and method used

5.1 scikit-learn

Object / method	Module	What it does (internally)	How it is used here
GradientBoostingRegressor	sklearn.ensemble	Builds the boosted tree ensemble with the algorithm in Section 3.2	the model; random_state=SEED
.fit(X, y)		Initialises F_0, then loops over M stages: residuals → subsample → fit tree → shrink and add	called by GridSearchCV for each fold and for the final refit

Object / method	Module	What it does (internally)	How it is used here
.predict(X)		Returns $F_0 + \nu \cdot \Sigma$ h_m(X) by dropping each row down every tree	pred_tr, pred_te
.staged_predict(X)		Generator giving the prediction after 1, 2, …, M trees	Figures 3–4, Table 4 (report)
.feature_importances_		Normalised impurity decrease per feature over all trees	printed and saved to gbr_split_params.csv
.init_, .estimators_, .oob_improvement_		Fitted F_0 (DummyRegressor); array of M DecisionTreeRegressor objects; OOB loss gain per stage	F_0 value, tree plots, Table 3, Figure 4 (report)
DecisionTreeRegressor	sklearn.tree	Greedy recursive binary splitting with friedman_mse; created internally by GBR	base learner h_m
DummyRegressor	sklearn.dummy	Predicts a constant (mean)	created internally as F_0
GridSearchCV	sklearn.model_selection	Exhaustive search over a parameter grid with cross-validation, then refit	384 combos × 5 folds; n_jobs=-1
.best_params_, .best_score_, .best_estimator_, .cv_results_, .best_index_		Winning settings; its mean CV score; refit model; per-combination results	printed; CV SD from cv_results_['std_test_score']
KFold	sklearn.model_selection	Splits indices into k folds, optionally shuffled	5 folds, shuffle=True, random_state=SEED
plot_tree, export_text	sklearn.tree	Draw / print a fitted tree's splits and leaf values	Figure 2 and tree listing (report only)

5.2 NumPy, pandas, Matplotlib, SciPy

Function	Library	Purpose
read_csv, dropna(subset=…), reset_index	pandas	load the data; remove the 1 row missing Vf (n = 369)
np.random.default_rng(seed).permutation(n)	NumPy	random shuffle of row indices (same as the Currey notebook)
DataFrame.iloc[idx[:n_test]] / [idx[n_test:]]	pandas	split into 92 test and 277 train rows
np.column_stack([Ca/1000, Vf])	NumPy	build the n×2 feature matrix sklearn expects (curve_fit takes a tuple instead)
metrics(obs, pred) (user function)	NumPy	R^2, RMSE, MAE, bias (Section 5.3); copied unchanged from currey.ipynb
DataFrame.assign, pd.concat, to_csv	pandas	write gbr_split_params.csv and gbr_split_predictions.csv (train + test rows with E_pred and set label)
plt.subplots, scatter, plot, axhline, set, legend, suptitle, tight_layout, savefig	Matplotlib	parity and residual plots → gbr_split.png (150 dpi)
scipy.optimize.curve_fit	SciPy	report only: refits the Currey model on the same split for Table 2 (k, x, y match currey.ipynb exactly)

5.3 Quality metrics (same definitions as the Currey notebook)

With residual $r_i = E_{obs,i} - E_{pred,i}$ over n samples:

$R^2 = 1 - \Sigma r_i^2 / \Sigma(E_{obs,i} - \bar{E}_{obs})^2$ — fraction of variance explained (1 = perfect, 0 = no better than the mean)

$RMSE = \sqrt{(\Sigma r_i^2 / n)}$ — typical error in GPa; penalises large misses

$MAE = \Sigma |r_i| / n$ — average absolute error in GPa; less sensitive to outliers

$bias = \Sigma r_i / n$ — mean residual; positive = model under-predicts on average

Note: for squared-error boosting, training bias is essentially zero by construction (F_0 is the mean and every leaf value is a mean residual). With subsample < 1 it is only approximately zero (0.078 GPa here).

6. Strengths and limitations in this application

- **Strengths:** no physical form has to be assumed; trends and the Ca × Vf interaction are learned from the data; insensitive to units and scaling; importances show which composition variable matters; easily extended to many more compositional inputs.
- **Limitations:** on this data it is barely better than a 3-parameter physical model, while using 50 trees × ~4 leaves; it needs hyperparameter tuning and over-fits with the default settings; the output is a non-smooth staircase with no interpretable constants like Currey's exponents; it gives no parameter standard errors (curve_fit does); it cannot extrapolate beyond the measured composition range.

References

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.

Friedman, J. H. (2002). Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4), 367–378.

Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed., Ch. 10 (Boosting and Additive Trees). Springer.

Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

scikit-learn User Guide, “Ensembles: Gradient boosting”; API docs for GradientBoostingRegressor and GridSearchCV; examples “Gradient Boosting regression” and “Gradient Boosting regularization”. https://scikit-learn.org

Currey, J. D. (1988). The effect of porosity and mineral content on the Young's modulus of elasticity of compact bone. *Journal of Biomechanics*, 21(2), 131–139.